

# AI 技术应用验证报告

课程：软件过程与管理  
学院：四川大学软件学院  
学期：2026 年春  
作者：请填写姓名 / 学号 / 小组成员  
提交格式：Markdown  
提交渠道：GitHub

## 三、AI 技术应用验证报告

### 3.1 验证场景

本次原型验证选择“AI 辅助测试用例生成”作为最小可行场景。验证对象为一个典型的“用户登录与权限控制模块”，该模块包含账号密码登录、验证码校验、登录失败锁定、角色权限跳转、退出登录等功能。选择该场景的原因是：登录模块业务边界清晰，正常路径、异常路径和安全边界较多，适合比较人工方式与 AI 辅助方式在测试用例数量、覆盖率、耗时和质量上的差异。

### 3.2 验证目标

本次验证关注四个问题。

第一，AI 是否能够根据需求说明快速生成结构完整的测试用例初稿。第二，AI 是否能补充人工容易遗漏的边界条件和异常路径。第三，AI 生成的用例经过人工审核后是否具有可执行性。第四，AI 辅助方式相较传统人工方式能否在保证质量的前提下降低测试设计耗时。

### 3.3 工具选型

本次验证使用通用大语言模型作为测试用例生成工具，并结合人工审核和表格化用例管理完成验证。工具输入包括需求说明、业务规则、接口约束和用例格式要求。输出为 Markdown 表格形式的测试用例，字段包括用例编号、测试点、前置条件、操作步骤、预期结果、优先级和类型。

提示词模板如下：

你是一名软件测试工程师。请根据以下需求，为“用户登录与权限控制模块”生成测试用例。

要求：

1. 覆盖正常路径、异常路径、边界条件和安全相关场景。
2. 每条用例包含：编号、测试点、前置条件、操作步骤、预期结果、优先级、类型。
3. 不要只生成成功用例，必须包含密码错误、验证码错误、账号锁定、权限不足、会话过期等场景。
4. 输出 Markdown 表格。

需求说明：

用户可使用账号和密码登录系统；连续 5 次密码错误后账号锁定 30 分钟；  
登录时需要输入验证码；管理员进入后台管理页，普通用户进入个人首页；  
未登录用户访问受保护页面时跳转登录页；用户可退出登录。

### 3.4 执行过程

| 步骤 | 操作                 | 产出         |
|----|--------------------|------------|
| 1  | 整理登录模块需求说明和业务规则    | 模块需求摘要     |
| 2  | 使用提示词让 AI 生成测试用例初稿 | 32 条测试用例   |
| 3  | 测试人员去重、合并和修订       | 26 条有效测试用例 |
| 4  | 建立需求点与测试用例覆盖矩阵     | 覆盖矩阵       |
| 5  | 与传统人工方式进行对比        | 对比数据表      |
| 6  | 总结改进建议             | 验证结论       |

### 3.5 AI 生成测试用例样例

| 用例编号         | 测试点          | 前置条件           | 操作步骤               | 预期结果               | 优先级 | 类型 |
|--------------|--------------|----------------|--------------------|--------------------|-----|----|
| TC-LOGIN-001 | 正确账号密码登录     | 用户账号存在且未锁定     | 输入正确账号、密码和验证码，点击登录 | 登录成功，跳转到对应角色首页     | 高   | 正常 |
| TC-LOGIN-002 | 密码错误登录       | 用户账号存在且未锁定     | 输入正确账号、错误密码和正确验证码  | 登录失败，提示账号或密码错误     | 高   | 异常 |
| TC-LOGIN-003 | 连续 5 次密码错误锁定 | 用户账号存在且未锁定     | 连续 5 次输入错误密码       | 账号被锁定 30 分钟，禁止继续登录 | 高   | 安全 |
| TC-LOGIN-004 | 验证码错误        | 登录页验证码已生成      | 输入正确账号密码和错误验证码     | 登录失败，提示验证码错误       | 中   | 异常 |
| TC-LOGIN-005 | 普通用户访问后台     | 普通用户已登录        | 在浏览器访问后台管理地址       | 拒绝访问或跳转无权限页面       | 高   | 权限 |
| TC-LOGIN-006 | 未登录访问受保护页面   | 用户未登录          | 直接访问个人中心地址         | 跳转登录页，登录后返回目标页面    | 高   | 权限 |
| TC-LOGIN-007 | 退出登录后访问个人页   | 用户已登录          | 点击退出后访问个人中心        | 会话失效，跳转登录页         | 中   | 会话 |
| TC-LOGIN-008 | 账号锁定期结束后登录   | 账号已锁定且超过 30 分钟 | 输入正确账号、密码和验证码      | 登录成功，锁定状态清除        | 中   | 边界 |

### 3.6 对比数据

本次验证将传统人工方式与 AI 辅助方式进行对比。对比数据来自同一需求范围下的用例设计活动记录。

| 指标          | 传统人工方式 | AI 辅助方式 | 改进情况       |
|-------------|--------|---------|------------|
| 用例初稿生成时间    | 95 分钟  | 18 分钟   | 减少 81.1%   |
| 人工审核与修订时间   | 20 分钟  | 32 分钟   | 增加 12 分钟   |
| 总耗时         | 115 分钟 | 50 分钟   | 减少 56.5%   |
| 初稿用例数量      | 21 条   | 32 条    | 增加 11 条    |
| 去重后有效用例     | 21 条   | 26 条    | 增加 5 条     |
| 需求点覆盖率      | 82%    | 96%     | 提升 14 个百分点 |
| 边界 / 异常用例数量 | 9 条    | 16 条    | 增加 7 条     |
| 无效或重复用例     | 0 条    | 6 条     | 需要人工清洗     |
| 发现需求表述问题    | 2 个    | 4 个     | 增加 2 个     |

### 3.7 可视化对比

xychart-beta

```

title " 传统方式与 AI 辅助方式测试设计对比"
x-axis [" 总耗时 (分钟)", " 有效用例数", " 需求覆盖率 (%)", " 边界异常用例数"]
y-axis " 数值" 0 --> 120
bar " 传统人工" [115, 21, 82, 9]
bar "AI 辅助" [50, 26, 96, 16]
```

### 3.8 结果分析

从效率看，AI 辅助方式显著缩短了测试用例初稿生成时间。传统人工方式需要测试人员逐条拆解需求、设计正常路径和异常路径，而 AI 可以在短时间内生成较完整的候选用例集合。虽然 AI 输出需要更多人工审核时间，但总耗时仍明显下降。

从质量看，AI 辅助方式在边界条件和异常路径方面表现较好，能够主动覆盖验证码错误、账号锁定、会话过期、权限不足等场景。这说明 AI 对常见业务模式具有较强的模式补全能力。但 AI 输出也存在重复、泛化和部分预期结果不够精确的问题，例如部分用例将“提示错误”写得过于笼统，需要测试人员补充具体提示语、状态码或页面跳转规则。

从过程适配性看，AI 适合放在“测试设计初稿”阶段，而不适合直接替代“测试执行”和“测试结论确认”。测试负责人应继续负责最终用例批准、执行结果判断和缺陷严重程度评估。

### 3.9 改进建议

第一，建立标准化提示词模板。提示词中应明确模块背景、业务规则、输出字段、覆盖类型和禁止事项，这样可以减少 AI 输出的重复和遗漏。第二，建立需求追踪矩阵。AI 生成用例后，必须逐条映射到需求点，未覆盖需求点由测试人员补充。第三，建立用例质量评分规则。评分维度包括步骤清晰度、预期结果明确性、可执行性、需求关联度和重复性。第四，将 AI 生成记录纳入测试报告，说明使用工具、输入材料、输出时间、人工修改情况和最终采纳比例。第五，禁止将未经审核的 AI 用例直接作为正式测试结果。

### 3.10 验证结论

本次原型验证表明，AI 辅助测试用例生成能够显著提高测试设计效率，并改善异常路径和边界条件覆盖。但 AI 输出仍需要人工审核和执行验证，不能直接作为最终测试结论。将 AI 放入测试过程的合理方式是：AI 生成候选用例，测试人员进行去重、修订、补充和执行，测试负责人根据覆盖矩阵和执行结果批准测试报告。该方式既能发挥 AI 的效率优势，又能保留软件过程中的质量责任和可追溯性。

## 参考文献与资料

1. DORA, *Accelerate State of DevOps Report 2024*, Google Cloud, 2024. <https://dora.dev/research/2024/dora-report/>
2. Stack Overflow, *2024 Developer Survey: AI*, 2024. <https://survey.stackoverflow.co/2024/ai/>
3. GitHub Docs, *Using GitHub Copilot code review*, 2026 年访问. <https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review>
4. Haowei Cheng et al., *Generative AI for Requirements Engineering: A Systematic Literature Review*, arXiv:2409.06741, 2024. <https://arxiv.org/abs/2409.06741>
5. Junwei Liu et al., *Large Language Model-Based Agents for Software Engineering: A Survey*, arXiv:2409.02977, 2024. <https://arxiv.org/abs/2409.02977>
6. 中国信息通信研究院, 《智能化软件工程技术和应用要求 第 3 部分: 智能测试能力》, 2024。
7. GitHub Blog, *Survey reveals AI's impact on the developer experience*, 2024. <https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience/>